AI Assisted Coding Lab 17.3

Veekshitha Adharasani

2303A52175

Batch – 41

Question 1: Employee Dataset Cleaning

Prompt:

Create an Employee information dataset. Th e task is to clean the created employee information dataset. It should handle missing values in salary, designation and experience. It should remove duplicate employee IDs. It should standardize text values like job titles in lowecase. Encode the categorical features like designation, employment_type. I need a clean dataset suitable for HR analytics and prediction tasks.

Code:

```python
import sys
try:
    import pandas as pd
except ModuleNotFoundError:
    print("Error: pandas is not installed. Install it with 'pip install pandas' and rerun the script.")
    sys.exit(1)
def create_employee_dataset() -> pd.DataFrame:
    data = {
        "employee_id": [101, 102, 103, 104, 105, 106, 106, 107, 108, 109, 110],
        "name": [
            "Anita Sharma",
            "Ravi Kumar",
            "Priya Singh",
            "Suresh Gupta",
            "Neha Patel",
            "Amit Roy",
            "Amit Roy",
            "Kavita Joshi",
            "Vijay Nair",
            "Deepa Menon",
            "Arjun Rao",
        ],
        "designation": [
            "Data Analyst",
            "data scientist",
            None,
            "HR Executive",
            "Finance Manager",
            "data engineer",
            "data engineer",
            "HR executive",
            "finance manager",
            "Data Analyst",
            "Business Analyst",
        ],
        "salary": [52000, None, 61000, 41000, 75000, 68000, 68000, 43000, 72000, None, 54000],
        "experience": [2, 5, None, 3, 8, 6, 6, None, 7, 4, 3],
```

```python
def create_employee_dataset() -> pd.DataFrame:
        "employment_type": [
            "Full-Time",
            "full-time",
            "Contract",
            "Part-Time",
            "full-time",
            "Contract",
            "Contract",
            None,
            "full-time",
            "Part-Time",
            "full-time",
        ],
        "department": [
            "Analytics",
            "Analytics",
            "Analytics",
            "Human Resources",
            "Finance",
            "Analytics",
            "Analytics",
            "Human Resources",
            "Finance",
            "Analytics",
            "Business",
        ],
    }
    return pd.DataFrame(data)
def clean_employee_dataset(df: pd.DataFrame) -> pd.DataFrame:
    cleaned = df.copy()
    cleaned = cleaned.drop_duplicates(subset=["employee_id"], keep="first")
    text_columns = ["designation", "employment_type", "department"]
    for col in text_columns:
        cleaned[col] = (
            cleaned[col]
            .astype(str)
```

```python
def create_employee_dataset() -> pd.DataFrame:
    return pd.DataFrame(data)
def clean_employee_dataset(df: pd.DataFrame) -> pd.DataFrame:
    cleaned = df.copy()
    cleaned = cleaned.drop_duplicates(subset=["employee_id"], keep="first")
    text_columns = ["designation", "employment_type", "department"]
    for col in text_columns:
        cleaned[col] = (
            cleaned[col]
            .astype(str)
            .str.strip()
            .str.lower()
            .replace({"none": None})
        )
    if cleaned["designation"].isna().any():
        designation_mode = cleaned["designation"].mode(dropna=True)
        if not designation_mode.empty:
            cleaned["designation"] = cleaned["designation"].fillna(designation_mode.iloc[0])
        else:
            cleaned["designation"] = cleaned["designation"].fillna("unknown")
    if cleaned["employment_type"].isna().any():
        employment_mode = cleaned["employment_type"].mode(dropna=True)
        if not employment_mode.empty:
            cleaned["employment_type"] = cleaned["employment_type"].fillna(employment_mode.iloc[0])
        else:
            cleaned["employment_type"] = cleaned["employment_type"].fillna("full-time")
    median_by_designation = cleaned.groupby("designation")["salary"].median()
    cleaned["salary"] = cleaned.apply(
        lambda row: median_by_designation[row["designation"]]
        if pd.isna(row["salary"]) and row["designation"] in median_by_designation
        else row["salary"],
        axis=1,
    )
    cleaned["salary"] = cleaned["salary"].fillna(cleaned["salary"].median())
    median_exp_by_designation = cleaned.groupby("designation")["experience"].median()
    cleaned["experience"] = cleaned.apply(
        lambda row: median_exp_by_designation[row["designation"]]
```

```python
def clean_employee_dataset(df: pd.DataFrame) -> pd.DataFrame:
        axis=1,
    )
    cleaned["experience"] = cleaned["experience"].fillna(cleaned["experience"].median())
    designation_mapping = {
        label: idx
        for idx, label in enumerate(sorted(cleaned["designation"].dropna().unique()), start=1)
    }
    employment_mapping = {
        label: idx
        for idx, label in enumerate(sorted(cleaned["employment_type"].dropna().unique()), start=1)
    }
    cleaned["designation_encoded"] = cleaned["designation"].map(designation_mapping)
    cleaned["employment_type_encoded"] = cleaned["employment_type"].map(employment_mapping)
    cleaned = cleaned[
        [
            "employee_id",
            "name",
            "designation",
            "designation_encoded",
            "employment_type",
            "employment_type_encoded",
            "department",
            "salary",
            "experience",
        ]
    ]
    return cleaned
def main() -> None:
    df = create_employee_dataset()
    cleaned_df = clean_employee_dataset(df)
    print("Original dataset:\n", df)
    print("\nCleaned dataset:\n", cleaned_df)
    cleaned_df.to_csv("clean_employee_dataset.csv", index=False)
    print("\nSaved cleaned dataset to clean_employee_dataset.csv")
if __name__ == "__main__":
    main()
```

Output:

```
Cleaned dataset:
    employee_id          name       designation  ...       department   salary  experience
0           101  Anita Sharma      data analyst  ...        analytics  52000.0         2.0
1           102    Ravi Kumar    data scientist  ...        analytics  56500.0         5.0
2           103   Priya Singh      data analyst  ...        analytics  61000.0         3.0
3           104  Suresh Gupta      hr executive  ...  human resources  41000.0         3.0
4           105    Neha Patel   finance manager  ...          finance  75000.0         8.0
5           106     Amit Roy     data engineer  ...        analytics  68000.0         6.0
7           107  Kavita Joshi      hr executive  ...  human resources  43000.0         3.0
8           108    Vijay Nair   finance manager  ...          finance  72000.0         7.0
9           109   Deepa Menon      data analyst  ...        analytics  56500.0         4.0
10          110     Arjun Rao  business analyst  ...         business  54000.0         3.0

[10 rows x 9 columns]

    employee_id          name       designation  ...       department   salary  experience
0           101  Anita Sharma      data analyst  ...        analytics  52000.0         2.0
1           102    Ravi Kumar    data scientist  ...        analytics  56500.0         5.0
2           103   Priya Singh      data analyst  ...        analytics  61000.0         3.0
3           104  Suresh Gupta      hr executive  ...  human resources  41000.0         3.0
4           105    Neha Patel   finance manager  ...          finance  75000.0         8.0
5           106     Amit Roy     data engineer  ...        analytics  68000.0         6.0
7           107  Kavita Joshi      hr executive  ...  human resources  43000.0         3.0
8           108    Vijay Nair   finance manager  ...          finance  72000.0         7.0
9           109   Deepa Menon      data analyst  ...        analytics  56500.0         4.0
10          110     Arjun Rao  business analyst  ...         business  54000.0         3.0

[10 rows x 9 columns]

3           104  Suresh Gupta      hr executive  ...  human resources  41000.0         3.0
4           105    Neha Patel   finance manager  ...          finance  75000.0         8.0
5           106     Amit Roy     data engineer  ...        analytics  68000.0         6.0
7           107  Kavita Joshi      hr executive  ...  human resources  43000.0         3.0
8           108    Vijay Nair   finance manager  ...          finance  72000.0         7.0
9           109   Deepa Menon      data analyst  ...        analytics  56500.0         4.0
10          110     Arjun Rao  business analyst  ...         business  54000.0         3.0

[10 rows x 9 columns]
```

```
4           105    Neha Patel   finance manager  ...          finance  75000.0         8.0
5           106     Amit Roy     data engineer  ...        analytics  68000.0         6.0
7           107  Kavita Joshi      hr executive  ...  human resources  43000.0         3.0
8           108    Vijay Nair   finance manager  ...          finance  72000.0         7.0
9           109   Deepa Menon      data analyst  ...        analytics  56500.0         4.0
10          110     Arjun Rao  business analyst  ...         business  54000.0         3.0

[10 rows x 9 columns]

3           104  Suresh Gupta      hr executive  ...  human resources  41000.0         3.0
4           105    Neha Patel   finance manager  ...          finance  75000.0         8.0
5           106     Amit Roy     data engineer  ...        analytics  68000.0         6.0
7           107  Kavita Joshi      hr executive  ...  human resources  43000.0         3.0
8           108    Vijay Nair   finance manager  ...          finance  72000.0         7.0
9           109   Deepa Menon      data analyst  ...        analytics  56500.0         4.0
10          110     Arjun Rao  business analyst  ...         business  54000.0         3.0

[10 rows x 9 columns]

7           107  Kavita Joshi      hr executive  ...  human resources  43000.0         3.0
8           108    Vijay Nair   finance manager  ...          finance  72000.0         7.0
9           109   Deepa Menon      data analyst  ...        analytics  56500.0         4.0
10          110     Arjun Rao  business analyst  ...         business  54000.0         3.0

[10 rows x 9 columns]

9           109   Deepa Menon      data analyst  ...        analytics  56500.0         4.0
10          110     Arjun Rao  business analyst  ...         business  54000.0         3.0

[10 rows x 9 columns]

Saved cleaned dataset to clean_employee_dataset.csv
PS D:\veekshitha\AI_Coding> w
```

Explanation:

This script makes a sample employee table and cleans it for HR use. It removes duplicate employee IDs, fixes missing pay, job title, and experience values, and converts text labels like job title and employment type into numbers. It also standardizes job titles and departments to lowercase so the data is consistent. Finally, it saves the cleaned result as a CSV file.

Question 2: Student Performance Data Preprocessing

Prompt:

Generate python code for preprocessing student performance data. Create a student performance dataset. Convert exam_data column into detetime format and extract the year, month, and day into separate columns. Normalize total_marks column using Min-Max scaling. Detect and handle outliers in the marks using IQR or Z-score. The final outpuut should be a transformed dataset with time based academic features and normalized marks.

Code:

```python
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
# ======================================================================
# STUDENT PERFORMANCE DATA PREPROCESSING
# ======================================================================
def create_student_dataset():
    """Create a sample student performance dataset"""
    np.random.seed(42)
    n_students = 100
    data = {
        'student_id': range(1001, 1001 + n_students),
        'student_name': [f'Student_{i}' for i in range(1, n_students + 1)],
        'exam_date': [datetime(2024, 1, 1) + timedelta(days=int(x))
                      for x in np.random.randint(0, 365, n_students)],
        'total_marks': np.random.normal(loc=75, scale=15, size=n_students),
        'attendance': np.random.uniform(70, 100, n_students),
        'subject': np.random.choice(['Math', 'Science', 'English', 'History'], n_students)
    }
    # Add some outliers intentionally
    data['total_marks'][5] = 120
    data['total_marks'][10] = 5
    data['total_marks'][15] = 130
    df = pd.DataFrame(data)
    return df
def extract_datetime_features(df):
    """Convert exam_date to datetime and extract year, month, day"""
    print("\n" + "="*70)
    print("STEP 1: Extracting DateTime Features")
    print("="*70)
    df['exam_date'] = pd.to_datetime(df['exam_date'])
    df['exam_year'] = df['exam_date'].dt.year
    df['exam_month'] = df['exam_date'].dt.month
    df['exam_day'] = df['exam_date'].dt.day
    df['exam_dayofweek'] = df['exam_date'].dt.day_name()
    df['exam_quarter'] = df['exam_date'].dt.quarter
    print(f"Original exam_date column type: {df['exam_date'].dtype}")
```

```python
def extract_datetime_features(df):
    return df
def detect_outliers_iqr(df, column, multiplier=1.5):
    """Detect outliers using IQR method"""
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - multiplier * IQR
    upper_bound = Q3 + multiplier * IQR
    outliers = df[(df[column] < lower_bound) | (df[column] > upper_bound)]
    return outliers, lower_bound, upper_bound
def detect_outliers_zscore(df, column, threshold=3):
    """Detect outliers using Z-score method"""
    z_scores = np.abs((df[column] - df[column].mean()) / df[column].std())
    outliers = df[z_scores > threshold]
    return outliers
def handle_outliers(df, column, method='iqr', action='clip'):
    """
    Detect and handle outliers
    Methods: 'iqr' or 'zscore'
    Actions: 'clip', 'remove', 'impute'
    """
    print("\n" + "="*70)
    print(f"STEP 2: Detecting and Handling Outliers ({method.upper()} Method)")
    print("="*70)
    df_processed = df.copy()
    if method == 'iqr':
        outliers, lower_bound, upper_bound = detect_outliers_iqr(df_processed, column)
        print(f"\nIQR Method Results:")
        print(f"Q1: {df[column].quantile(0.25):.2f}")
        print(f"Q3: {df[column].quantile(0.75):.2f}")
        print(f"IQR: {df[column].quantile(0.75) - df[column].quantile(0.25):.2f}")
        print(f"Lower Bound: {lower_bound:.2f}")
        print(f"Upper Bound: {upper_bound:.2f}")
        print(f"Outliers detected: {len(outliers)}")
        if len(outliers) > 0:
            print(f"\nOutlier records:")
```

```python
def handle_outliers(df, column, method='iqr', action='clip'):
        elif action == 'remove':
            df_processed = df_processed[~df_processed.index.isin(outliers.index)]
            print(f"\n{len(outliers)} rows removed")
        elif action == 'impute':
            median_val = df_processed[column].median()
            df_processed.loc[outliers.index, column] = median_val
            print(f"\nOutliers replaced with median: {median_val:.2f}")
    return df_processed
def normalize_marks(df, column):
    """Normalize total_marks using Min-Max scaling"""
    print("\n" + "="*70)
    print("STEP 3: Normalizing Marks using Min-Max Scaling")
    print("="*70)
    print(f"\nOriginal {column} statistics:")
    print(f"Min: {df[column].min():.2f}")
    print(f"Max: {df[column].max():.2f}")
    print(f"Mean: {df[column].mean():.2f}")
    print(f"Std Dev: {df[column].std():.2f}")
    min_val = df[column].min()
    max_val = df[column].max()
    df[f'{column}_normalized'] = ((df[column] - min_val) / (max_val - min_val)) * 100
    print(f"\nMin-Max Scaling Formula: (X - {min_val:.2f}) / ({max_val:.2f} - {min_val:.2f}) * 100")
    print(f"\nNormalized statistics (0-100 scale):")
    print(f"Min: {df[f'{column}_normalized'].min():.2f}")
    print(f"Max: {df[f'{column}_normalized'].max():.2f}")
    print(f"Mean: {df[f'{column}_normalized'].mean():.2f}")
    print(f"\nSample - Original vs Normalized:")
    print(df[['student_id', column, f'{column}_normalized']].head(10).to_string(index=False))
    return df
def create_academic_features(df):
    """Create time-based academic features"""
    print("\n" + "="*70)
    print("STEP 4: Creating Time-Based Academic Features")
    print("="*70)
    df['performance_category'] = pd.cut(df['total_marks_normalized'],
                                        bins=[0, 40, 60, 80, 100],
```

```python
def save_datasets(df_original, df_processed):
    df_processed.to_csv(processed_path, index=False)
    print("\n" + "="*70)
    print("DATASETS SAVED")
    print("="*70)
    print(f"Original: {original_path}")
    print(f"Processed: {processed_path}")
def main():
    """Main execution"""
    print("\n" + "="*70)
    print("STUDENT PERFORMANCE DATA PREPROCESSING PIPELINE")
    print("="*70)
    df_original = create_student_dataset()
    print(f"\nDataset created with {len(df_original)} students")
    print(df_original.head(10))
    df = df_original.copy()
    df = extract_datetime_features(df)
    df = handle_outliers(df, 'total_marks', method='iqr', action='clip')
    df = normalize_marks(df, 'total_marks')
    df = create_academic_features(df)
    generate_summary_statistics(df_original, df)
    save_datasets(df_original, df)
    print("\n" + "="*70)
    print("FINAL PROCESSED DATASET")
    print("="*70)
    print(df[[
        'student_id', 'exam_date', 'exam_year', 'exam_month', 'exam_day',
        'total_marks', 'total_marks_normalized', 'performance_category',
        'semester', 'combined_score'
    ]].head(15).to_string(index=False))
    print("\n" + "="*70)
    print("PREPROCESSING COMPLETED SUCCESSFULLY")
    print("="*70)
    return df_original, df
if __name__ == "__main__":
    df_original, df_processed = main()
```

Output:

```
PS D:\veekshitha\AI_Coding> & C:/Users/Anveekshith/AppData/Local/Programs/Python/Python313/python.exe d:/veekshitha/AI_Coding/Lab_17_3.py

 ======================================================================
 STUDENT PERFORMANCE DATA PREPROCESSING PIPELINE
 ======================================================================

 Dataset created with 100 students
    student_id student_name  exam_date  total_marks  attendance  subject
 0        1001    Student_1 2024-04-12    83.731842   77.255569  Science
 1        1002    Student_2 2024-12-14    88.316227   72.793083  Science
 2        1003    Student_3 2024-09-27    88.414985   96.916473  Science
 3        1004    Student_4 2024-04-16    86.324967   97.012542     Math
 4        1005    Student_5 2024-03-12    71.892512   88.993844  History
 5        1006    Student_6 2024-07-07   120.000000   80.170894  Science
 6        1007    Student_7 2024-01-21    52.377701   80.476287  Science
 7        1008    Student_8 2024-04-12    91.494705   91.778670  English
 8        1009    Student_9 2024-05-01    72.334018   96.913308  Science
 9        1010   Student_10 2024-08-02    68.844250   96.612593  English

 ======================================================================
 STEP 1: Extracting DateTime Features
 ======================================================================
 Original exam_date column type: datetime64[us]

 Extracted datetime features:
    exam_date  exam_year  exam_month  exam_day exam_dayofweek
 0 2024-04-12       2024           4        12         Friday
 1 2024-12-14       2024          12        14       Saturday
 2 2024-09-27       2024           9        27         Friday
 3 2024-04-16       2024           4        16        Tuesday
 4 2024-03-12       2024           3        12        Tuesday
 5 2024-07-07       2024           7         7         Sunday
 6 2024-01-21       2024           1        21         Sunday
 7 2024-04-12       2024           4        12         Friday
 8 2024-05-01       2024           5         1      Wednesday
 9 2024-08-02       2024           8         2         Friday

 ======================================================================
 STEP 2: Detecting and Handling Outliers (IQR Method)
```

```
max    130.000000  114.279133

Performance Distribution:
performance_category
Poor         33
Average      35
Good         25
Excellent     5
Name: count, dtype: int64

================================================================
DATASETS SAVED
================================================================
Original: d:\veekshitha\AI_Coding\student_performance_original.csv
Processed: d:\veekshitha\AI_Coding\student_performance_processed.csv

================================================================
FINAL PROCESSED DATASET
================================================================
 student_id  exam_date  exam_year  exam_month  exam_day  total_marks  total_marks_normalized performance_category semester  combined_score
       1001 2024-04-12       2024           4        12    83.731842               58.492243             Average   Summer       64.121241
       1002 2024-12-14       2024          12        14    88.316227               64.721520                Good   Winter       67.142989
       1003 2024-09-27       2024           9        27    88.414985               64.855713                Good     Fall       74.473941
       1004 2024-04-16       2024           4        16    86.324967               62.015790                Good   Summer       72.514815
       1005 2024-03-12       2024           3        12    71.892512               42.404923             Average   Spring       56.381359
       1006 2024-07-07       2024           7         7   114.279133              100.000000           Excellent     Fall       94.051268
       1007 2024-01-21       2024           1        21    52.377701               15.888136                Poor   Spring       35.264581
       1008 2024-04-12       2024           4        12    91.494705               69.040446                Good   Summer       75.861913
       1009 2024-05-01       2024           5         1    72.334018               43.004844             Average   Summer       59.177383
       1010 2024-08-02       2024           8         2    68.844250               38.262936                Poor     Fall       55.767833
       1011 2024-11-26       2024          11        26    40.684958                0.000000                 NaN   Winter       28.018880
       1012 2024-03-28       2024           3        28    61.526881               28.320071                Poor   Spring       46.602335
       1014 2024-12-25       2024          12        25    79.448421               52.671917             Average   Winter       59.325000
       1015 2024-05-31       2024           5        31    59.432552               25.474290                Poor   Summer       46.918991

================================================================
PREPROCESSING COMPLETED SUCCESSFULLY
================================================================
PS D:\veekshitha\AI_Coding>
```

Explanation:

This Python code creates a sample student performance dataset with exam dates, marks, and attendance. It converts exam dates to datetime format and extracts year, month, and day. The code detects outliers in marks using the IQR method, clips them to valid ranges, and normalizes marks to a 0-100 scale. Finally, it adds time-based features like performance categories and semesters, then saves both original and processed datasets as CSV files.

Question 3: Student Health Data Preparation

Prompt:

Generate a python code to clean and preprocess student health records. Create a Student Health records dataset. Make sure the code handles missing values in height, weight, BMI, encides categorical features like medical_condition, fitness_status, scales numerical features using standard scaling, splits dataset into training and testing set. Finally, the output should be a preprocesssed dataset for predictive health analysis.

Code:

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split
# ====================================================================
# STUDENT HEALTH RECORDS PREPROCESSING
# ====================================================================
def create_health_dataset():
    """Create a sample student health records dataset"""
    np.random.seed(42)
    n_students = 200
    data = {
        'student_id': range(1001, 1001 + n_students),
        'student_name': [f'Student_{i}' for i in range(1, n_students + 1)],
        'age': np.random.randint(18, 25, n_students),
        'gender': np.random.choice(['Male', 'Female'], n_students),
        'height': np.random.normal(170, 10, n_students),  # cm
        'weight': np.random.normal(65, 15, n_students),   # kg
        'BMI': None,  # Will calculate later
        'medical_condition': np.random.choice(['None', 'Asthma', 'Diabetes', 'Hypertension', 'Allergy'], n_students),
        'fitness_status': np.random.choice(['Poor', 'Average', 'Good', 'Excellent'], n_students),
        'blood_pressure_systolic': np.random.normal(120, 15, n_students),
        'blood_pressure_diastolic': np.random.normal(80, 10, n_students),
        'cholesterol_level': np.random.normal(200, 30, n_students)
    }
    df = pd.DataFrame(data)
    # Calculate BMI
    df['BMI'] = df['weight'] / ((df['height'] / 100) ** 2)
    # Introduce missing values
    df.loc[np.random.choice(df.index, 20, replace=False), 'height'] = np.nan
    df.loc[np.random.choice(df.index, 15, replace=False), 'weight'] = np.nan
    df.loc[np.random.choice(df.index, 10, replace=False), 'BMI'] = np.nan
    return df
def handle_missing_values(df):
    """Handle missing values in height, weight, BMI"""
    print("\n" + "="*70)
    print("STEP 1: Handling Missing Values")
```

```python
def encode_categorical_features(df):
    """Encode categorical features"""
    print("\n" + "="*70)
    print("STEP 2: Encoding Categorical Features")
    print("="*70)
    # Label encode gender
    le_gender = LabelEncoder()
    df['gender_encoded'] = le_gender.fit_transform(df['gender'])
    # One-hot encode medical_condition
    medical_dummies = pd.get_dummies(df['medical_condition'], prefix='medical')
    df = pd.concat([df, medical_dummies], axis=1)
    # One-hot encode fitness_status
    fitness_dummies = pd.get_dummies(df['fitness_status'], prefix='fitness')
    df = pd.concat([df, fitness_dummies], axis=1)
    print("Categorical features encoded:")
    print("- gender: Label encoded")
    print("- medical_condition: One-hot encoded")
    print("- fitness_status: One-hot encoded")
    return df
def scale_numerical_features(df):
    """Scale numerical features using Standard Scaling"""
    print("\n" + "="*70)
    print("STEP 3: Scaling Numerical Features")
    print("="*70)
    numerical_cols = ['age', 'height', 'weight', 'BMI', 'blood_pressure_systolic',
                      'blood_pressure_diastolic', 'cholesterol_level']
    print(f"Numerical features to scale: {numerical_cols}")
    scaler = StandardScaler()
    df_scaled = df.copy()
    df_scaled[numerical_cols] = scaler.fit_transform(df[numerical_cols])
    print("\nScaling applied using StandardScaler (Z-score normalization)")
    print("Formula: (X - mean) / std")
    return df_scaled, scaler
def split_dataset(df, test_size=0.2, random_state=42):
    """Split dataset into training and testing sets"""
    print("\n" + "="*70)
    print("STEP 4: Splitting Dataset")
```

```python
def split_dataset(df, test_size=0.2, random_state=42):
    df['target'] = df['fitness_status'].apply(lambda x: 1 if x in ['Good', 'Excellent'] else 0)
    feature_cols = [col for col in df.columns if col not in ['student_id', 'student_name', 'gender',
                                                             'medical_condition', 'fitness_status', 'target']]
    X = df[feature_cols]
    y = df['target']
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_size,
                                                        random_state=random_state, stratify=y)
    print(f"Dataset split: {len(X_train)} training samples, {len(X_test)} testing samples")
    print(f"Feature columns: {len(feature_cols)}")
    return X_train, X_test, y_train, y_test, feature_cols
def generate_summary_statistics(df_original, df_processed):
    """Generate summary statistics"""
    print("\n" + "="*70)
    print("SUMMARY STATISTICS")
    print("="*70)
    print(f"\nOriginal dataset shape: {df_original.shape}")
    print(f"Processed dataset shape: {df_processed.shape}")
    print("\nNumerical features statistics (after scaling):")
    numerical_cols = ['age', 'height', 'weight', 'BMI', 'blood_pressure_systolic',
                      'blood_pressure_diastolic', 'cholesterol_level']
    print(df_processed[numerical_cols].describe())
    print("\nCategorical distributions:")
    print("Medical conditions:")
    print(df_original['medical_condition'].value_counts())
    print("\nFitness status:")
    print(df_original['fitness_status'].value_counts())
def save_preprocessed_data(df_processed, X_train, X_test, y_train, y_test):
    """Save preprocessed datasets"""
    base_path = 'd:\\veekshitha\\AI_Coding\\'
    df_processed.to_csv(f'{base_path}student_health_processed.csv', index=False)
    X_train.to_csv(f'{base_path}student_health_X_train.csv', index=False)
    X_test.to_csv(f'{base_path}student_health_X_test.csv', index=False)
    y_train.to_csv(f'{base_path}student_health_y_train.csv', index=False)
    y_test.to_csv(f'{base_path}student_health_y_test.csv', index=False)
    print("\n" + "="*70)
```

```python
    print(f"Testing target: {base_path}student_health_y_test.csv")
def main():
    """Main preprocessing pipeline"""
    print("\n" + "="*70)
    print("STUDENT HEALTH RECORDS PREPROCESSING PIPELINE")
    print("="*70)
    # Step 1: Create dataset
    print("\nCreating student health records dataset...")
    df_original = create_health_dataset()
    print(f"Dataset created with {len(df_original)} students")
    print("\nFirst 10 records:")
    print(df_original.head(10))
    df = df_original.copy()
    # Step 2: Handle missing values
    df = handle_missing_values(df)
    # Step 3: Encode categorical features
    df = encode_categorical_features(df)
    # Step 4: Scale numerical features
    df, scaler = scale_numerical_features(df)
    # Step 5: Split dataset
    X_train, X_test, y_train, y_test, feature_cols = split_dataset(df)
    # Step 6: Generate summary
    generate_summary_statistics(df_original, df)
    # Step 7: Save datasets
    save_preprocessed_data(df, X_train, X_test, y_train, y_test)
    print("\n" + "="*70)
    print("FINAL PROCESSED DATASET (First 15 records)")
    print("="*70)
    display_cols = ['student_id', 'age', 'gender_encoded', 'height', 'weight', 'BMI',
                    'blood_pressure_systolic', 'fitness_Good', 'fitness_Excellent', 'target']
    print(df[display_cols].head(15).to_string(index=False))
    print("\n" + "="*70)
    print("PREPROCESSING COMPLETED SUCCESSFULLY")
    print("="*70)
    print("Ready for predictive health analysis!")
    return df, X_train, X_test, y_train, y_test, scaler
if __name__ == "__main__":
```

Output:

```
 ====================================================================
STUDENT HEALTH RECORDS PREPROCESSING PIPELINE
 ====================================================================

Creating student health records dataset...
Dataset created with 200 students

First 10 records:
   student_id student_name  age  gender      height     weight        BMI  \
0        1001    Student_1   24  Female  156.699259  26.534870  10.806451
1        1002    Student_2   21    Male  169.747124  82.140888  28.507199
2        1003    Student_3   22    Male  183.431682  36.548792  10.862363
3        1004    Student_4   24  Female  162.654524  73.986400  27.965308
4        1005    Student_5   20  Female  160.401054  62.478938  24.283943
5        1006    Student_6   22  Female         NaN  69.277630  21.134862
6        1007    Student_7   22  Female  176.342403  57.921204  18.626192
7        1008    Student_8   24  Female  200.610952        NaN  16.960744
8        1009    Student_9   19  Female  156.814298  57.921837  23.554344
9        1010   Student_10   20  Female  177.203872  56.657283        NaN

  medical_condition fitness_status  blood_pressure_systolic  \
0           Allergy        Average                134.785841
1      Hypertension           Poor                111.083032
2          Diabetes        Average                123.262619
3           Allergy           Good                149.188069
4           Allergy           Good                130.374310
5           Allergy      Excellent                139.292227
6          Diabetes        Average                132.072048
7          Diabetes           Good                133.953800
8            Asthma        Average                120.596125
9      Hypertension           Good                120.170227
```

```
 ====================================================================
STEP 1: Handling Missing Values
 ====================================================================
Missing values before imputation:
height    20
weight    15
BMI       10
dtype: int64

Missing values after imputation:
height    0
weight    0
BMI       0
dtype: int64

 ====================================================================
STEP 2: Encoding Categorical Features
 ====================================================================
Categorical features encoded:
- gender: Label encoded
- medical_condition: One-hot encoded
- fitness_status: One-hot encoded

 ====================================================================
STEP 3: Scaling Numerical Features
 ====================================================================
Numerical features to scale: ['age', 'height', 'weight', 'BMI', 'blood_pressure_systolic', '

Scaling applied using StandardScaler (Z-score normalization)
Formula: (X - mean) / std

 ====================================================================
```

```
Name: count, dtype: int64

...  ==================================================================
     DATASETS SAVED
     ==================================================================
     Processed dataset: d:\veekshitha\AI_Coding\student_health_processed.csv
     Training features: d:\veekshitha\AI_Coding\student_health_X_train.csv
     Testing features: d:\veekshitha\AI_Coding\student_health_X_test.csv
     Training target: d:\veekshitha\AI_Coding\student_health_y_train.csv
     Testing target: d:\veekshitha\AI_Coding\student_health_y_test.csv

     ==================================================================
     FINAL PROCESSED DATASET (First 15 records)
     ==================================================================
      student_id       age  gender_encoded        height    weight       BMI  blood_pressure_systolic  fitness_Good  fitness_Excellent  target
            1001  1.416577               0 -1.370992e+00 -2.533204 -2.027410                 0.803337         False              False       0
            1002 -0.085095               1 -1.871895e-02  1.017056  0.895084                -0.694666         False              False       0
            1003  0.415462               1  1.399541e+00 -1.893848 -2.018179                 0.075076         False              False       0
            1004  1.416577               0 -7.537919e-01  0.496419  0.805615                 1.713550          True              False       1
            1005 -0.585652               0 -9.873402e-01 -0.238294  0.197801                 0.524531          True              False       1
            1006  0.415462               0 -2.945609e-15  0.195780 -0.322131                 1.088138         False               True       1
            1007  0.415462               0  6.648119e-01 -0.529290 -0.736326                 0.631827         False              False       0
            1008  1.416577               0  3.179990e+00  0.061729 -1.011301                 0.750753          True              False       1
            1009 -1.086209               0 -1.359069e+00 -0.529250  0.077340                -0.093444         False              False       0
            1010 -0.585652               0  7.540941e-01 -0.609987 -0.832611                -0.120361          True              False       1
            1011  1.416577               0  3.348765e-01 -0.936616 -0.973508                 0.122042         False              False       0
            1012 -0.585652               1  6.892777e-02  1.383955  1.174558                 0.565136          True              False       1
            1013 -0.585652               0  9.511385e-01  0.061729 -0.182862                -0.399700         False              False       0
            1014  0.415462               0 -2.331819e-02  0.123303  0.095011                -0.015413          True              False       1
            1015 -0.085095               1 -1.999360e+00 -0.915858 -0.037714                -0.425324         False               True       1
```

```
      1013 -0.585652               0  9.511385e-01  0.061729 -0.182862                -0.399700         False              False       0
      1014  0.415462               0 -2.331819e-02  0.123303  0.095011                -0.015413          True              False       1
      1015 -0.085095               1 -1.999360e+00 -0.915858 -0.037714                -0.425324         False               True       1

 ==================================================================
 PREPROCESSING COMPLETED SUCCESSFULLY
 ==================================================================
 Ready for predictive health analysis!
 /tmp/ipykernel_4202/1919739225.py:42: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
 The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

 For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the c


   df['height'].fillna(df['height'].mean(), inplace=True)
 /tmp/ipykernel_4202/1919739225.py:44: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
 The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

 For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the c


   df['weight'].fillna(df['weight'].median(), inplace=True)
 /tmp/ipykernel_4202/1919739225.py:46: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
 The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

 For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the c


   df['BMI'].fillna(df['weight'] / ((df['height'] / 100) ** 2), inplace=True)
```

Explanation:

This Python code creates a sample student health dataset with missing values in height, weight, and BMI. It fills missing values using mean/median imputation, encodes categorical features like medical conditions and fitness status, and scales numerical features with standard scaling. Finally, it splits the data into training and testing sets for predictive health analysis, saving all processed datasets as CSV files.

Question 4: Real-Time Application: Ride-Sharing Data Cleaning

Prompt:

Create an Employee information dataset. Th e task is to clean the created employee information dataset. It should handle missing values in salary, designation and experience. It should remove duplicate employee IDs. It should

standardize text values like job titles in lowecase. Encode the categorical features like designation, employment_type. I need a clean dataset suitable for HR analytics and prediction tasks.

Code:

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
# ======================================================================
# RIDESHARE TRIP DATA PREPROCESSING
# ======================================================================
def create_rideshare_dataset():
    """Creates a sample ride-sharing trip dataset with common data quality issues."""
    np.random.seed(42)
    n_trips = 200
    data = {
        'trip_id': range(10001, 10001 + n_trips),
        'pickup_location': np.random.choice([
            'Times Square, NYC',
            'Time Square, New York',
            'Grand Central Station',
            'Grand Central',
            'Midtown West',
            'midtown west',
            'Brooklyn Downtown',
            'Brooklyn Dtwn',
            'JFK Airport',
            'JFK Airport, Queens'
        ], n_trips),
        'dropoff_location': np.random.choice([
            'Times Square, NYC',
            'Time Square, New York',
            'Grand Central Station',
            'Grand Central',
            'Midtown West',
            'midtown west',
            'Brooklyn Downtown',
            'Brooklyn Dtwn',
            'LGA Airport',
            'LaGuardia Airport, Queens'
        ], n_trips),
        'trip_distance': np.random.normal(5, 3, n_trips).clip(0.5, 20),
```

```python
def standardize_locations(df):
    """Standardizes pickup and dropoff location names."""
    print("\n" + "="*70)
    print("STEP 1: Standardizing Locations")
    print("="*70)
    for col in ['pickup_location', 'dropoff_location']:
        df[col] = df[col].str.lower().str.strip()
        df[col] = df[col].replace({
            'time square, new york': 'times square, nyc',
            'grand central': 'grand central station',
            'midtown west': 'midtown west, nyc',
            'brooklyn dtwn': 'brooklyn downtown',
            'jfk airport, queens': 'jfk airport',
            'laguardia airport, queens': 'lga airport'
        })
    print("Locations standardized (lowercase, stripped, and common aliases mapped).")
    return df
def fill_missing_fares(df):
    """Fills missing fare amounts with the median."""
    print("\n" + "="*70)
    print("STEP 2: Filling Missing Fare Amounts")
    print("="*70)
    print(f"Missing 'fare_amount' before imputation: {df['fare_amount'].isnull().sum()}")
    median_fare = df['fare_amount'].median()
    df['fare_amount'] = df['fare_amount'].fillna(median_fare)
    print(f"Missing 'fare_amount' after imputation: {df['fare_amount'].isnull().sum()} (filled with median: {median_far
    return df
def remove_duplicates(df):
    """Removes duplicate ride entries based on key identifiers."""
    print("\n" + "="*70)
    print("STEP 3: Removing Duplicate Ride Entries")
    print("="*70)
    initial_rows = len(df)
    # Define columns to consider for identifying duplicates
    duplicate_cols = ['pickup_location', 'dropoff_location', 'trip_distance', 'fare_amount', 'driver_id', 'trip_timesta
    df = df.drop_duplicates(subset=duplicate_cols)
```

```python
def normalize_driver_ratings(df):
    """Normalizes driver ratings to a 0-1 scale using MinMaxScaler, filling missing with mean before scaling."""
    print("\n" + "="*70)
    print("STEP 4: Normalizing Driver Ratings")
    print("="*70)
    print(f"Missing 'driver_rating' before imputation: {df['driver_rating'].isnull().sum()}")
    # Fill missing ratings with the mean before scaling
    mean_rating = df['driver_rating'].mean()
    df['driver_rating'] = df['driver_rating'].fillna(mean_rating)
    print(f"Missing 'driver_rating' after imputation: {df['driver_rating'].isnull().sum()} (filled with mean: {mean_rating:.2f})")
    scaler = MinMaxScaler()
    df['driver_rating_normalized'] = scaler.fit_transform(df[['driver_rating']])
    print("Driver ratings normalized to a 0-1 scale using MinMaxScaler.")
    print(f"Original min rating: {df['driver_rating'].min():.2f}, max rating: {df['driver_rating'].max():.2f}")
    print(f"Normalized min rating: {df['driver_rating_normalized'].min():.2f}, max rating: {df['driver_rating_normalized'].max():.2f}")
    return df
def generate_summary(df_original, df_processed):
    """Generates a summary of data improvements."""
    print("\n" + "="*70)
    print("SUMMARY OF DATA IMPROVEMENTS")
    print("="*70)
    print(f"Original dataset shape: {df_original.shape}")
    print(f"Processed dataset shape: {df_processed.shape}")
    print(f"Rows removed: {len(df_original) - len(df_processed)}")
    print("\nMissing values before vs. after:")
    initial_missing = df_original[['fare_amount', 'driver_rating']].isnull().sum()
    final_missing = df_processed[['fare_amount', 'driver_rating']].isnull().sum()
    print("  'fare_amount': initial = {}\054 final = {}".format(initial_missing['fare_amount'], final_missing['fare_amount']))
    print("  'driver_rating': initial = {}\054 final = {}".format(initial_missing['driver_rating'], final_missing['driver_rating']))
    print("\nLocation standardization examples (first 5 unique):")
    original_locations = df_original['pickup_location'].str.lower().str.strip().unique()
    processed_locations = df_processed['pickup_location'].unique()
    print(f"  Original unique pickup locations: {len(original_locations)}")
    print(f"  Processed unique pickup locations: {len(processed_locations)}")
    print("  Original (sample): ", sorted(original_locations)[:5])
    print("  Processed (sample): ", sorted(processed_locations)[:5])
```

```python
def generate_summary(df_original, df_processed):
    print(f"  Processed Mean: {df_processed['driver_rating'].mean():.2f}, Median: {df_processed['driver_rating'].median():.2f}")
    print(f"  Normalized Driver Rating (min, max): ({df_processed['driver_rating_normalized'].min():.2f}, {df_processed['driver_rating_normalized'
def main_rideshare_preprocessing():
    """Main preprocessing pipeline for ride-sharing trip data."""
    print("\n" + "="*70)
    print("RIDESHARE TRIP DATA PREPROCESSING PIPELINE")
    print("="*70)
    print("\nCreating sample dataset...")
    df_original = create_rideshare_dataset()
    print(f"Initial dataset created with {len(df_original)} trips.")
    print("\nFirst 5 records of original data:")
    display(df_original.head())
    df = df_original.copy()
    # Step 1: Standardize pickup and drop locations
    df = standardize_locations(df)
    # Step 2: Fill missing fare amounts
    df = fill_missing_fares(df)
    # Step 3: Remove duplicate ride entries
    df = remove_duplicates(df)
    # Step 4: Normalize driver ratings
    df = normalize_driver_ratings(df)
    # Step 5: Generate summary of improvements
    generate_summary(df_original, df)
    print("\n" + "="*70)
    print("FINAL PROCESSED DATASET (First 10 records)")
    print("="*70)
    display_cols = ['trip_id', 'pickup_location', 'dropoff_location', 'trip_distance',
                    'fare_amount', 'driver_id', 'driver_rating', 'driver_rating_normalized',
                    'payment_method', 'trip_timestamp']
    display(df[display_cols].head(10))
    print("\n" + "="*70)
    print("PREPROCESSING COMPLETED SUCCESSFULLY")
    print("="*70)
    print("Clean dataset ready for operational analysis!")
    return df
```

Output:

The code defines a complete preprocessing pipeline for ride-sharing trip data. It includes functions for:

1. **`create_rideshare_dataset()`** : Generates a sample dataset with common data issues like inconsistent location names, missing values for `fare_amount` and `driver_rating`, and duplicate entries.
2. **`standardize_locations()`** : Cleans `pickup_location` and `dropoff_location` columns by converting them to lowercase, stripping whitespace, and mapping common aliases to a standardized form.
3. **`fill_missing_fares()`** : Imputes missing values in the `fare_amount` column with the median of the existing values.
4. **`remove_duplicates()`** : Identifies and removes duplicate ride entries based on a subset of key columns (`pickup_location`, `dropoff_location`, `trip_distance`, `fare_amount`, `driver_id`, `trip_timestamp`).
5. **`normalize_driver_ratings()`** : Fills missing `driver_rating` values with the mean and then normalizes the ratings to a 0-1 scale using `MinMaxScaler`.
6. **`generate_summary()`** : Provides a detailed overview of the data improvements, comparing the original and processed datasets regarding shape, removed rows, missing values, and location standardization.
7. **`main_rideshare_preprocessing()`** : Orchestrates the entire pipeline, calling each preprocessing function in sequence and displaying intermediate and final results.

The final output is a `DataFrame` (`df_processed`) that is cleaned and ready for further operational analysis.

```
======================================================================
RIDESHARE TRIP DATA PREPROCESSING PIPELINE
======================================================================

Creating sample dataset...
Initial dataset created with 210 trips.

First 5 records of original data:
```

	trip_id	pickup_location	dropoff_location	trip_distance	fare_amount	driver_id	driver_rating	payment_method	trip_timestamp
0	10001	Brooklyn Downtown	midtown west	5.551567	24.697251	134	3.1	Credit Card	2023-01-13 17:33:00
1	10002	Grand Central	Times Square, NYC	6.733691	13.847606	124	4.1	Credit Card	2023-01-17 01:16:00
2	10003	Brooklyn Dtwn	LGA Airport	11.403924	16.024329	125	3.7	Apple Pay	2023-01-11 18:45:00
3	10004	Midtown West	midtown west	5.680546	11.149293	110	3.9	Credit Card	2023-01-17 05:58:00
4	10005	Brooklyn Downtown	Grand Central Station	9.966385	5.815561	137	4.6	Credit Card	2023-01-23 19:37:00

```
======================================================================
STEP 1: Standardizing Locations
======================================================================
Locations standardized (lowercase, stripped, and common aliases mapped).

======================================================================
STEP 2: Filling Missing Fare Amounts
======================================================================
Missing 'fare_amount' before imputation: 17
Missing 'fare_amount' after imputation: 0 (filled with median: 15.44)

======================================================================
STEP 3: Removing Duplicate Ride Entries
======================================================================
Removed 10 duplicate rows.

======================================================================
STEP 4: Normalizing Driver Ratings
======================================================================
Missing 'driver_rating' before imputation: 5
Missing 'driver_rating' after imputation: 0 (filled with mean: 3.80)
Driver ratings normalized to a 0-1 scale using MinMaxScaler.
Original min rating: 2.50, max rating: 5.00
Normalized min rating: 0.00, max rating: 1.00

======================================================================
SUMMARY OF DATA IMPROVEMENTS
======================================================================
```

```
Processed (sample): ['brooklyn downtown', 'grand central station', 'jfk airport', 'midtown west, nyc', 'times square, nyc']

Driver Rating statistics:
  Original Mean: 3.80, Median: 3.80
  Processed Mean: 3.80, Median: 3.80
  Normalized Driver Rating (min, max): (0.00, 1.00)

======================================================================
FINAL PROCESSED DATASET (First 10 records)
======================================================================
/tmp/ipykernel_4202/2351689336.py:83: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the o


  df['fare_amount'].fillna(median_fare, inplace=True)
/tmp/ipykernel_4202/2351689336.py:108: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the o


  df['driver_rating'].fillna(mean_rating, inplace=True)
```

	trip_id	pickup_location	dropoff_location	trip_distance	fare_amount	driver_id	driver_rating	driver_rating_normalized	payment_method	trip_timestamp
0	10001	brooklyn downtown	midtown west, nyc	5.551567	24.697251	134	3.1	0.24	Credit Card	2023-01-13 17:33:00
1	10002	grand central station	times square, nyc	6.733691	13.847606	124	4.1	0.64	Credit Card	2023-01-17 01:16:00
2	10003	brooklyn downtown	lga airport	11.403924	16.024329	125	3.7	0.48	Apple Pay	2023-01-11 18:45:00
3	10004	midtown west, nyc	midtown west, nyc	5.680546	11.149293	110	3.9	0.56	Credit Card	2023-01-17 05:58:00
4	10005	brooklyn downtown	grand central station	9.966385	5.815561	137	4.6	0.84	Credit Card	2023-01-23 19:37:00
6	10006	jfk airport	grand central station	3.611224	15.441247	101	3.5	0.40	Credit Card	2023-01-08 04:41:00
7	10007	grand central station	grand central station	5.259697	5.000000	106	4.4	0.76	Cash	2023-01-12 08:49:00
8	10008	brooklyn downtown	grand central station	3.221352	22.316844	117	3.6	0.44	Apple Pay	2023-01-03 02:41:00
9	10009	brooklyn downtown	lga airport	2.959987	5.000000	126	3.4	0.36	Apple Pay	2023-01-01 01:45:00
10	10010	midtown west, nyc	grand central station	4.209782	31.257140	133	4.9	0.96	Cash	2023-01-07 07:18:00

```
======================================================================
PREPROCESSING COMPLETED SUCCESSFULLY
======================================================================
Clean dataset ready for operational analysis!
```

Explanation:

This code sets up a complete data cleaning process for ride-sharing trip information. It creates a sample dataset, then fixes common issues like inconsistent location names, missing fares, duplicate entries, and unstandardized driver ratings. The goal is to provide a clean, ready-to-use dataset for analysis.